

# Post-Signup Issue - Root Cause Analysis & Fix

**Date:** January 6, 2026

**Status:** ROOT CAUSE IDENTIFIED

**Severity:** CRITICAL BLOCKER

**Impact:** Users cannot access dashboard after signup

---

## Problem Summary

After completing the 5-step signup process, users are redirected to `/home` but the page displays a loading spinner indefinitely. The dashboard never loads, making the application completely unusable for new users.

---

## Root Cause Analysis

### The Issue Chain

1. **User completes signup** → Demographics step → `navigate('/home')`
2. **Frontend redirects to** `/home` → Home component mounts
3. **Home component calls** `useSession()` → Checks for session cookie
4. **Session context calls** `api.auth.getUser()` → GET `/api/auth/me`
5. **Backend** `/api/auth/me` **endpoint checks** `req.session.userId`
6. **Session middleware initializes session from PostgreSQL session store**
7. **Session store lookup fails** → Session data not found in database
8. `req.session.userId` **is undefined** → Returns 401 "Not authenticated"
9. **Frontend receives 401** → Clears session cookie
10. **Home component sees** `user === null` → Stays in loading state
11. **Infinite loop** → Page never loads

### Why Session Store Lookup Fails

The issue is in the **session store initialization** on the backend:

```
TypeScript
```

```
// server/index.js, lines 46-60
app.use(session({
  store: new PgStore({
    pool,
    createTableIfMissing: true, // ← This creates the table IF MISSING
  }),
  secret: env.SESSION_SECRET,
  // ...
}));
```

**Problem:** The `connect-pg-simple` session store creates a table called `session` in PostgreSQL. However:

1. **The table might not exist** on the live server
2. **The table schema might be incorrect** (wrong columns or types)
3. **The database connection might be failing silently**
4. **The `pool` object might not be properly initialized**

When the session store cannot find or create the table, it silently fails and session data is never persisted. This means:

- Session is created in memory during signup
- Session data is returned to frontend (signup succeeds)
- But when frontend calls `/api/auth/me`, the session store can't find the session
- Backend returns 401
- Frontend clears cookie and shows loading state

## Evidence

From signup flow (server/routes/auth.js, lines 179-194):

TypeScript

```
// Set session with guardian verification status
req.session.userId = newUser.id;
req.session.email = newUser.email;
req.session.age = userAge;
req.session.requiresGuardianVerification = requiresGuardianVerification;
req.session.guardianVerifiedAt = requiresGuardianVerification ? null : new
Date().toISOString();

res.status(201).json({
  user: {
    id: newUser.id,
```

```
    email: newUser.email,
    age: userAge,
    requiresGuardianVerification,
    guardianVerifiedAt: req.session.guardianVerifiedAt,
  }
});
```

The signup endpoint sets `req.session.userId` and returns successfully. But if the session store fails to persist this to the database, the next request won't have access to it.

**From `getUser` endpoint (server/routes/auth.js, lines 307-340):**

TypeScript

```
router.get('/me', async (req, res) => {
  try {
    if (!req.session.userId) { // ← This will be undefined if session
wasn't persisted
      return res.status(401).json({ error: 'Not authenticated' });
    }
    // ... rest of endpoint
  }
});
```

## Solution

### Fix 1: Verify Session Store Configuration

**File:** `server/index.js`

TypeScript

```
import pgSession from 'connect-pg-simple';

const PgStore = pgSession(session);

async function createServer() {
  const app = express();

  // ... other middleware ...

  // Initialize session store with error handling
  const store = new PgStore({
    pool,
    createTableIfMissing: true,
```

```

    tableName: 'session', // Explicitly set table name
    errorHandler: (err) => {
        logger.error({ err }, 'Session store error - falling back to memory
store');
        // Note: In production, this should alert ops team
    }
});

// Verify session table exists
try {
    await pool.query(`
        CREATE TABLE IF NOT EXISTS "session" (
            "sid" varchar NOT NULL COLLATE "default",
            "sess" json NOT NULL,
            "expire" timestamp(6) NOT NULL,
            PRIMARY KEY ("sid")
        ) WITH (OIDS=FALSE);
        CREATE INDEX IF NOT EXISTS "IDX_session_expire" on "session"
("expire");
    `);
    logger.info('Session table verified/created successfully');
} catch (err) {
    logger.error({ err }, 'Failed to create session table');
}

app.use(session({
    store: store,
    secret: env.SESSION_SECRET,
    resave: false,
    saveUninitialized: false,
    cookie: {
        secure: env.NODE_ENV === 'production',
        httpOnly: true,
        sameSite: 'strict',
        maxAge: 1000 * 60 * 60 * 24 * 7, // 7 days
    }
})));

// ... rest of server setup ...
}

```

## Fix 2: Add Session Debugging Middleware

**File:** server/middleware/sessionDebug.ts

TypeScript

```

import { Request, Response, NextFunction } from 'express';
import logger from '../logger';

export function sessionDebugMiddleware(req: Request, res: Response, next:
NextFunction) {
  // Log session info for debugging
  if (process.env.DEBUG_SESSION === 'true') {
    logger.debug({
      path: req.path,
      method: req.method,
      sessionId: req.sessionID,
      userId: req.session.userId,
      hasSession: !!req.session,
    }, 'Session debug');
  }

  // Ensure session is saved after response
  res.on('finish', () => {
    if (req.session && req.session.userId) {
      req.session.save((err) => {
        if (err) {
          logger.error({ err, userId: req.session.userId }, 'Failed to save
session');
        }
      });
    }
  });

  next();
}

```

Add to server setup:

TypeScript

```
app.use(sessionDebugMiddleware);
```

### Fix 3: Add Explicit Session Save After Signup

**File:** server/routes/auth.js , lines 179-194

TypeScript

```

// Set session with guardian verification status
req.session.userId = newUser.id;
req.session.email = newUser.email;

```

```

req.session.age = userAge;
req.session.requiresGuardianVerification = requiresGuardianVerification;
req.session.guardianVerifiedAt = requiresGuardianVerification ? null : new
Date().toISOString();

// CRITICAL: Explicitly save session to database
req.session.save((err) => {
  if (err) {
    logger.error({ err, userId: newUser.id }, 'Failed to save session after
signup');
    // Continue anyway - session may still be in memory
  }
});

res.status(201).json({
  user: {
    id: newUser.id,
    email: newUser.email,
    age: userAge,
    requiresGuardianVerification,
    guardianVerifiedAt: req.session.guardianVerifiedAt,
  }
});

```

## Fix 4: Add Health Check for Session Store

**File:** server/routes/health.js

TypeScript

```

import express from 'express';
import { pool } from '../db.js';
import logger from '../logger.ts';

const router = express.Router();

router.get('/', async (req, res) => {
  try {
    // Check database connection
    const result = await pool.query('SELECT NOW()');

    // Check session table exists
    const tableCheck = await pool.query(`
      SELECT EXISTS (
        SELECT FROM information_schema.tables
        WHERE table_name = 'session'
      );
    `);
  }
});

```

```

`);

const sessionTableExists = tableCheck.rows[0]?.exists || false;

res.json({
  status: 'ok',
  timestamp: new Date().toISOString(),
  database: {
    connected: true,
    sessionTableExists: sessionTableExists,
  }
});
} catch (err) {
  logger.error({ err }, 'Health check failed');
  res.status(503).json({
    status: 'error',
    error: 'Database connection failed',
    timestamp: new Date().toISOString(),
  });
}
});

export default router;

```

## Fix 5: Frontend Session Retry Logic

**File:** src/lib/session.tsx

TypeScript

```

const fetchUser = async (retries = 3) => {
  // Skip API call if no session cookie exists
  if (!hasSessionCookie()) {
    setUser(null);
    setLoading(false);
    return;
  }

  for (let attempt = 1; attempt <= retries; attempt++) {
    try {
      const { data, error } = await api.auth.getUser();
      if (!error && data?.user) {
        setUser(data.user);
        setLoading(false);
        return;
      } else {
        // Session expired or invalid

```

```

        clearSessionCookie();
        setUser(null);
        setLoading(false);
        return;
    }
} catch (error) {
    logger.warn({ attempt, error }, 'Failed to fetch user, retrying...');

    if (attempt < retries) {
        // Wait before retry (exponential backoff)
        await new Promise(resolve => setTimeout(resolve, 1000 * attempt));
    } else {
        // All retries exhausted
        logger.error({ error }, 'Failed to fetch user after retries');
        clearSessionCookie();
        setUser(null);
        setLoading(false);
        return;
    }
}
}
};

```

## Testing the Fix

### Step 1: Verify Session Table

Bash

```

# SSH into production server
ssh user@roomxi.live

# Connect to database
psql $DATABASE_URL

# Check if session table exists
\d session

# If not, create it manually
CREATE TABLE IF NOT EXISTS "session" (
    "sid" varchar NOT NULL COLLATE "default",
    "sess" json NOT NULL,
    "expire" timestamp(6) NOT NULL,
    PRIMARY KEY ("sid")
) WITH (OIDS=FALSE);

```



```
CREATE INDEX IF NOT EXISTS "IDX_session_expire" on "session" ("expire");
```

## Step 2: Enable Session Debugging

Bash

```
# Set debug flag
export DEBUG_SESSION=true

# Restart server
npm run start
```

## Step 3: Test Signup Flow

Bash

```
# Go to https://roomxi.live/auth/signup
# Complete all 5 steps
# Should redirect to /home and load dashboard
# Check server logs for session debug info
```

## Step 4: Verify Session Persistence

Bash

```
# Query session table to see if session was saved
psql $DATABASE_URL
SELECT sid, expire FROM "session" LIMIT 5;

# Should show recent sessions with future expiration dates
```

## Expected Behavior After Fix

1. User completes signup
2. Backend creates user in database
3. Backend sets `req.session.userId`
4. Backend **explicitly saves session** to PostgreSQL
5. Backend returns 201 with user data

6. Frontend receives response and stores in state
  7. Frontend redirects to `/home`
  8. Home component mounts and calls `useSession()`
  9. Session context calls `api.auth.getUser()`
  10. Backend retrieves session from PostgreSQL
  11. Backend returns user data
  12. Frontend renders dashboard
  13.  User sees their dashboard with mood orb
- 

## Verification Checklist

- ☐ Session table exists in PostgreSQL
  - ☐ Session table has correct schema (sid, sess, expire columns)
  - ☐ `connect-pg-simple` is properly configured
  - ☐ `req.session.save()` is called after signup
  - ☐ Health check endpoint returns `sessionTableExists: true`
  - ☐ Signup flow completes without errors
  - ☐ Dashboard loads after signup
  - ☐ Session persists across page refreshes
  - ☐ Logout clears session properly
- 

## Alternative Temporary Workaround

If the above fixes don't work immediately, implement a temporary workaround:

**File:** `src/routes/auth/Signup.tsx`

TypeScript

```
// After successful signup, redirect to /explore instead of /home
// This allows users to use the app while debugging session issue
navigate('/explore');

// Or show a message asking user to sign in
setTimeout(() => {
```

```
navigate('/auth/login');  
, 2000);
```

This is NOT a permanent solution but allows users to access the app while the session store issue is being debugged.

---

## Root Cause Prevention

To prevent this issue in the future:

1. **Add automated session table creation** during server startup
  2. **Add health checks** that verify session store is working
  3. **Add monitoring** for session store errors
  4. **Add integration tests** that verify signup → login flow works
  5. **Add pre-deployment checks** to ensure session table exists
  6. **Document session store configuration** in deployment guide
- 

**Status:** Ready for Implementation

**Priority:** CRITICAL - Implement immediately

**Estimated Time:** 30 minutes to 2 hours (depending on root cause)